VertiCore Complete End-to-End Architecture & Workflow Specification

1. Objective of This Document

This document presents a **complete, end-to-end, deeply detailed architecture and workflow specification** for VertiCore. It consolidates all components, execution paths, safety mechanisms, and workflows into a single, authoritative technical reference.

This document is intended for: - Platform architects - Senior engineers - Security & compliance teams - SRE / infrastructure teams

---

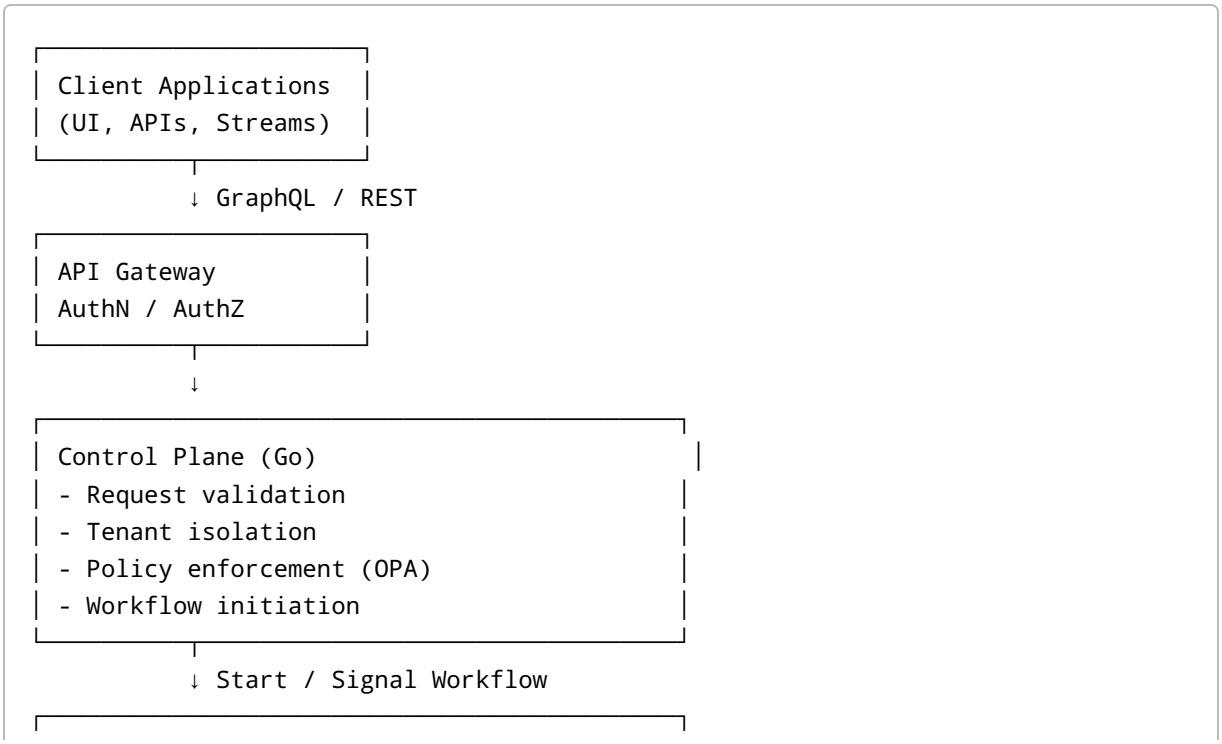
2. Core Architectural Philosophy

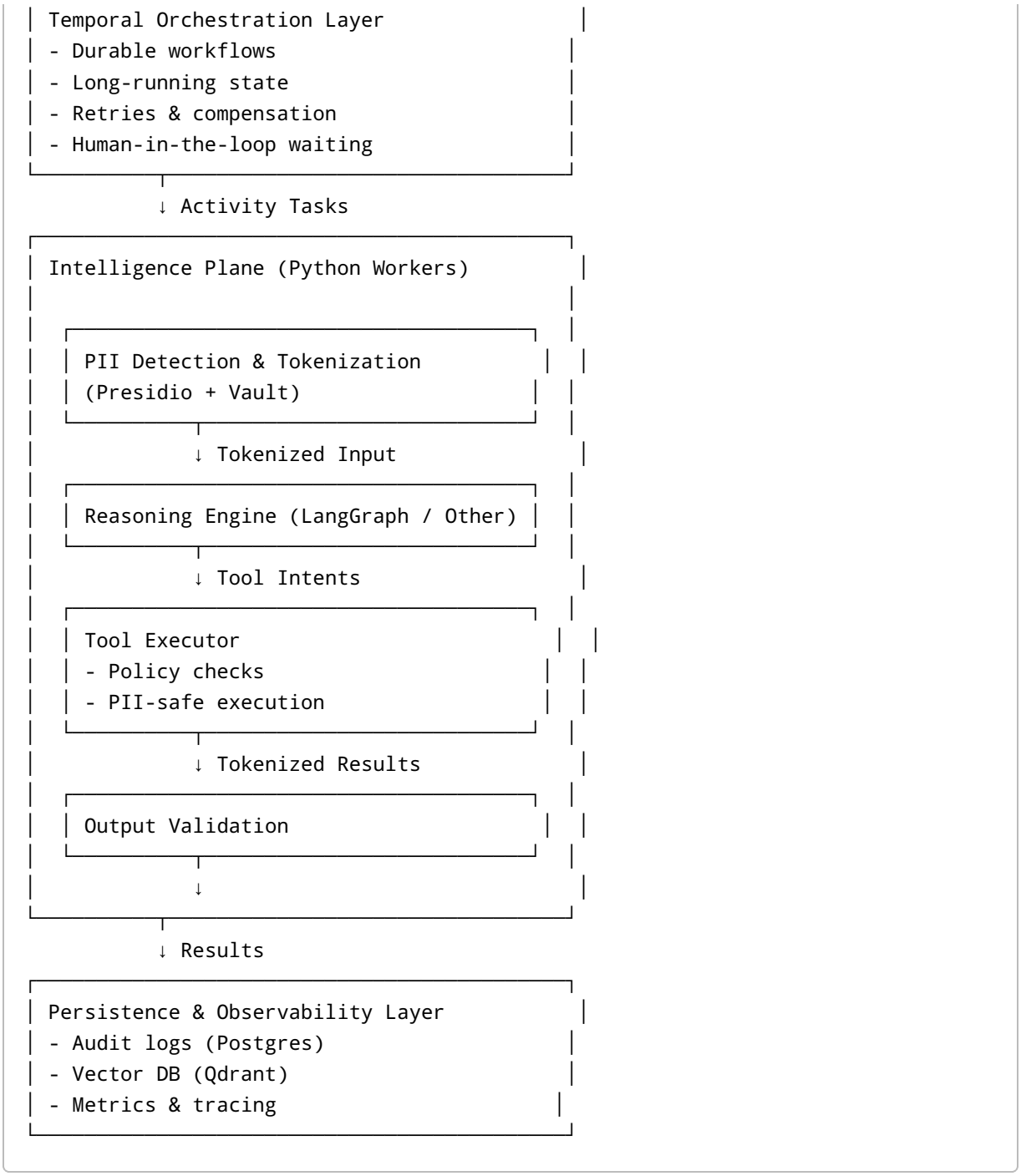
VertiCore is built on the principle that **AI reasoning must never control execution, state, or authority**.

The system explicitly separates: - **Reasoning** (LLM-based, probabilistic) - **Execution** (deterministic, durable) - **Governance** (policy, audit, compliance)

---

3. High-Level Architecture Overview





---

4. API & Ingress Layer

Responsibilities

- Client authentication (OAuth2 / mTLS)
- Request schema validation

- Rate limiting
- Tenant identification

All requests are converted into **strongly typed internal commands**.

---

5. Control Plane (Go)

Responsibilities

- Workflow routing
- Policy evaluation via OPA
- Enforcement of platform invariants

Key Guarantees

- Stateless
- Deterministic
- No LLM dependency

The Control Plane never executes AI logic.

---

6. Temporal Orchestration Layer

Core Responsibilities

- Start and coordinate workflows
- Maintain authoritative workflow state
- Handle retries, timeouts, and failures
- Pause/resume for human review

Workflow Semantics

- One workflow = one business transaction
- All state changes are explicit
- Replay produces identical outcomes

---

7. Intelligence Plane (Python Workers)

Worker Characteristics

- Stateless containers
- Horizontally scalable
- Ephemeral execution

Workers may crash at any time without data loss.

---

8. PII Sandwich (End-to-End)

Nested PII Model

```
Ingress Data
  → Detect & Tokenize
  → Reasoning Engine
     → Tool Intent
        → (Optional) Rehydrate
        → Tool Call
        → Detect & Tokenize
     → Reasoning Continues
  → Output Validation
  → (Optional) Final Rehydration
```

No raw PII reaches: - LLMs - Logs - Vector databases

---

9. Reasoning Engine Layer

Key Properties

- Stateless
- Deterministic routing
- Structured outputs

Supported Engines

- LangGraph (default)
- Rule engines
- State-machine engines

Reasoning engines emit **intent**, never side effects.

---

10. Tool Execution Layer

Tool Invocation Semantics

- Tools executed by platform, not reasoning engine
- Policy checks before execution
- Immediate response sanitization

Failure Handling

- Retries via Temporal
- Idempotent design required

---

11. Memory Architecture

Memory Layers

Layer	Owner	Purpose
Workflow State	Temporal	Authoritative memory
Working Memory	Reasoning Engine	Ephemeral
Knowledge Memory	Vector DB	Recall-only
Audit Memory	Postgres	Compliance

Memory promotion is explicit and policy-governed.

---

12. Human-in-the-Loop Workflow

Trigger Conditions

- Low confidence
- Missing data
- Policy violation

Execution Flow

1. Agent emits review intent
2. Temporal suspends workflow
3. Human action recorded
4. Workflow resumes
5. Fresh agent execution

---

13. Multi-Agent Workflow Patterns

Supported Patterns

- Sequential pipeline
- Fan-out / fan-in
- Supervisor pattern

- Human escalation

Agents never communicate directly.

---

14. Observability & Audit

Observability

- OpenTelemetry traces
- Node-level metrics

Audit

- Immutable event logs
- Tool execution records
- Policy decisions

---

15. Security & Threat Model

Threats Addressed

- Prompt injection
- Data leakage
- Rogue agent behavior

Mitigations

- Policy enforcement
- No autonomous loops
- Strict boundaries

---

16. Failure Modes & Recovery

Failure	Handling
Worker crash	Activity retry
Tool failure	Retry / fallback
Human delay	Durable wait

No partial state persists.

---

17. CI/CD & Validation

- Static graph validation
- Policy tests
- Replay tests

---

18. End-to-End Example: Mortgage Workflow

1. Application submitted
2. Workflow started
3. Documents analyzed
4. Tools invoked
5. Risk assessed
6. Human review (if needed)
7. Decision finalized
8. Audit persisted

---

19. Why This Architecture Works

This architecture makes **complexity explicit, bounded, and governable**, enabling safe AI systems for finance and healthcare.

---

20. Final Summary

VertiCore is not an agent framework — it is a **deterministic execution platform that safely embeds AI reasoning**.

Every component exists to enforce correctness, safety, and trust.